NovaCart

AI Chatbot & Agent Integration

NAME: ASFAND YAR KHAN

REG NO: SP23-BCS-153

Subject: E-Commerce Development
Project: NovaCart Online Grocery Store
Stack: React.js · Node.js · Express · MongoDB

Introduction

NovaCart is a full-stack E-Commerce grocery platform built with React.js on the frontend and Node.js/Express with MongoDB on the backend. As part of this assignment, an AI-powered chatbot named Nova has been integrated into the platform. Nova is capable of understanding natural-language messages and responding intelligently to help users shop, track orders, manage their cart, apply discount coupons, and get answers to frequently asked questions.

The chatbot is powered by a rule-based intent detection engine on the backend (chatbotController.js) and a dedicated React component (ChatBot.jsx) on the frontend. It communicates via a REST API at /api/chatbot/query and is accessible from every page through a floating chat trigger button.

1 Product Discovery & Search

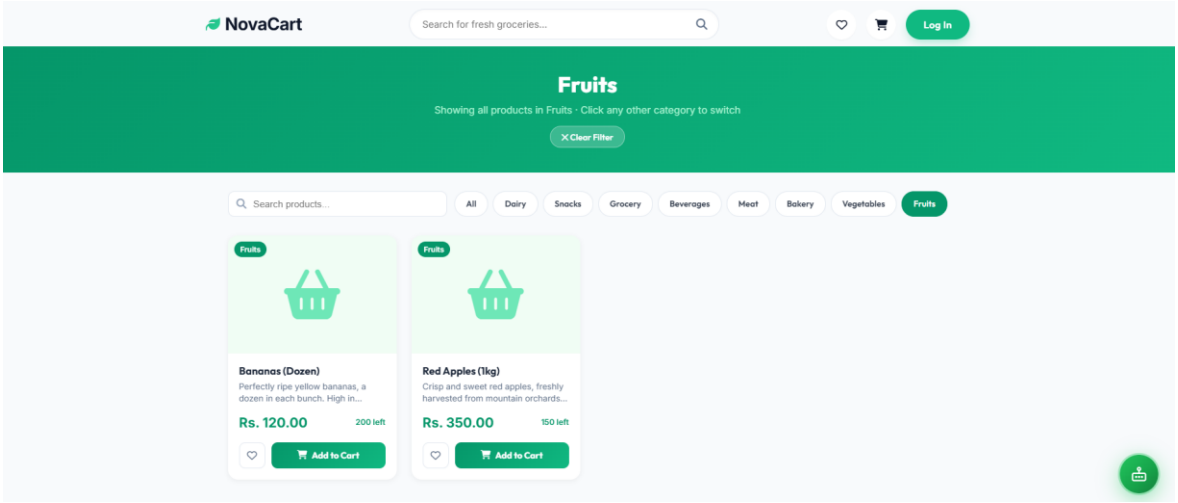
This feature allows users to search for products using plain English instead of filling out filter forms. The chatbot understands queries such as 'show me fruits under Rs500', 'find chicken products', or 'I want vegetables above Rs100'. It extracts three key pieces of information from the message:

- Category — using a keyword map (fruit, vegetable, dairy, bakery, meat, beverage, snack, grocery)
- Price range — extracted with regex patterns (under, above, between, max, min)
- General search term — leftover keywords after stripping filler words

▸ **Natural Language Search Intent Detection (Backend)**

```
// INTENTS defined in chatbotController.js
const INTENTS = {
  product_search: /\b(show|find|search|get|look|browse|display|
                   products?|items?|buy|shop|available|list|
                   recommend|suggest)\b/i,
  recommendation: /\b(recommend|suggestion|popular|trending|
                   best|top|rated|featured)\b/i,
};

// Price parser — extracts numeric range from free text
const parsePrice = (msg) => {
  const under = msg.match(
    /(?:under|below|less than|max)\s*₨?\s*(\d+)/i
  );
  const over  = msg.match(
    /(?:above|over|more than|min)\s*₨?\s*(\d+)/i
  );
  const between = msg.match(
    /between\s*₨?\s*(\d+)\s*(?:and|to|-)\s*₨?\s*(\d+)/i
  );
  if (between) return { min: Number(between[1]), max: Number(between[2]) };
  if (under)   return { max: Number(under[1]) };
  if (over)    return { min: Number(over[1]) };
```

```
  return null;
};
```



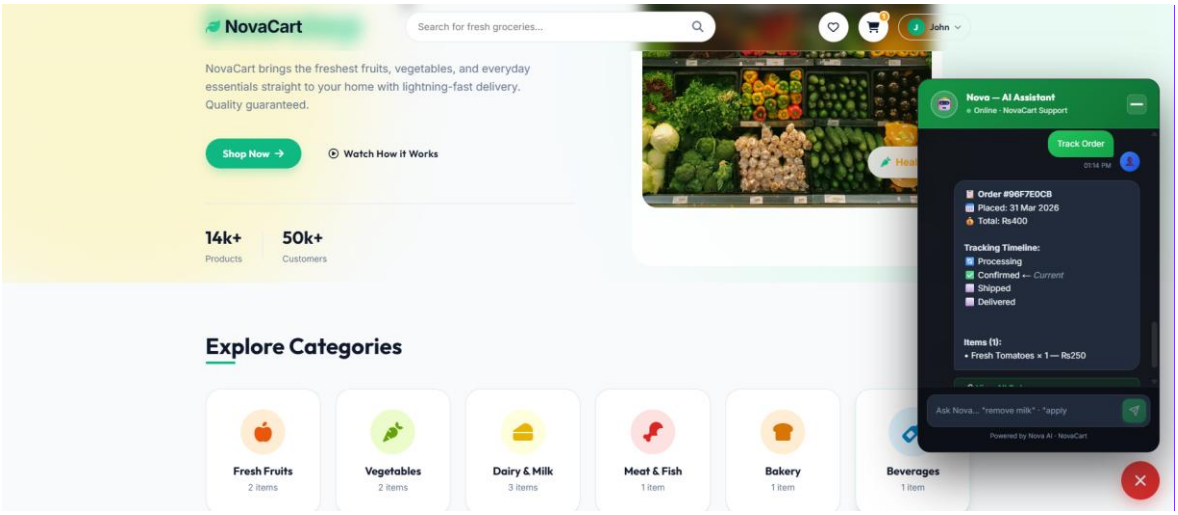
▸ MongoDB Query Construction

```
const query = {};
if (priceFilter?.min) query.price = { $gte: priceFilter.min };
if (priceFilter?.max) query.price = { ...query.price, $lte: priceFilter.max };
if (category) query.category = { $regex: category, $options: 'i' };
if (searchTerm) {
  query.$or = [
    { name:        { $regex: searchTerm, $options: 'i' } },
    { category:    { $regex: searchTerm, $options: 'i' } },
    { description: { $regex: searchTerm, $options: 'i' } },
  ];
}
const products = await Product.find(query).limit(6);
```

▸ Smart Autocomplete Suggestions

As the user types in the chat input box, the frontend calls /api/chatbot/suggestions with a debounce of 300ms. The backend looks up matching product names from MongoDB and also returns matching FAQ action keywords ('shipping info', 'track my order', etc.).

```
// Frontend — ChatBot.jsx
const handleInputChange = (e) => {
  const val = e.target.value;
  setInput(val);
  clearTimeout(suggestionsTimeout.current);
  if (val.length >= 2) {
    suggestionsTimeout.current = setTimeout(async () => {
      const results = await fetchSuggestions(val); // ChatAgent.js
      setSuggestions(results);
    }, 300);
  } else {
    setSuggestions([]);
  }
};
```



2 Product Recommendation Engine

Nova's recommendation system surfaces relevant products based on user intent and contextual signals. While the current implementation is rule-based (not ML-driven), it mimics an intelligent recommendation engine through several strategies:

- Trending / New Arrivals — When the user asks for 'trending', 'popular', or 'top products', the system fetches the most recently added products sorted by createdAt descending.
- Category-based — If the user asks for 'recommendations for dairy' or 'best snacks', the category is detected and used to filter results.
- "Customers Also Bought" style — After adding a product to cart via chat, the bot presents quick replies like 'Show similar products' or 'Browse [category]' to encourage discovery.
- Fallback browse — If no strong signal is detected, the shop page is surfaced.

▸ Trending Products Query

```
// chatbotController.js — Recommendation handler
const isTrending = INTENTS.recommendation.test(lower)
                && !INTENTS.product_search.test(lower);

const products = await Product.find(
    isTrending && Object.keys(query).length === 0 ? {} : query
  )
  .limit(6)
  .sort(isTrending ? { createdAt: -1 } : { price: priceFilter?.max ? 1 : -1 });
```

▸ Post-Add Recommendation Follow-up (Frontend)

```
// ChatBot.jsx — handleAddToCart
const handleAddToCart = (product) => {
  addToCart(product, 1);
  appendBotMessage({
    text: `✅ **${product.name}** added to your cart!\n
           Continue shopping or proceed to checkout.`,
    quickReplies: ['Go to cart', 'Continue shopping', 'Apply coupon'],
    action: { type: 'navigate', url: '/cart', label: '🛒 View Cart' }
```

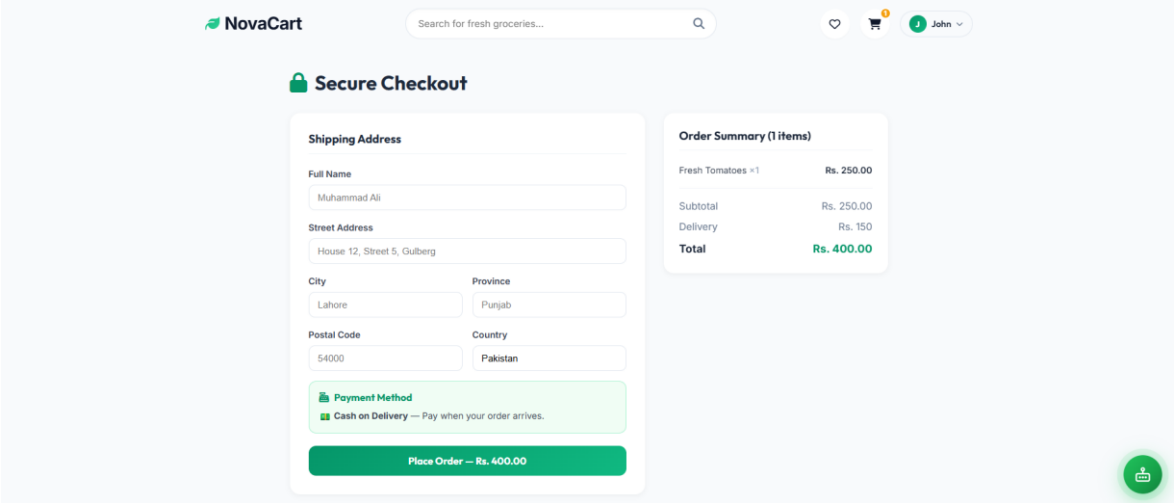
```
  });
};
```

3 Order Tracking

Users can ask the chatbot natural-language questions about their orders — 'Where is my order?', 'Track order #ABC123', or simply 'order status'. The system authenticates the request via JWT (optional auth middleware), then queries MongoDB for the user's most recent order or a specific order by ID.

▸ How It Works

- If the user is not logged in → bot responds with a login prompt and a navigation link.
- If a specific order ID (24-char hex) is detected in the message → that order is fetched directly.
- Otherwise → the 3 most recent orders are fetched; the latest one's timeline is displayed.
- Cancelled orders are clearly labelled ❌Order Cancelled.



▸ Order Timeline Builder (Backend)

```
// chatbotController.js
const formatOrderTimeline = (order) => {
  const steps  = ['processing', 'confirmed', 'shipped', 'delivered'];
  const icons  = ['□','✅','□','□'];
  const labels = ['Processing','Confirmed','Shipped','Delivered'];
  const statusIdx = steps.indexOf(order.status);

  let timeline = `□ **Order #${order._id.toString().slice(-
8).toUpperCase()}**\n`;
  timeline += `□ Placed: ${new Date(order.createdAt).toLocaleDateString()}\n`;
  timeline += `□ Total: ₨${order.total.toLocaleString()}\n\n`;
  timeline += `**Tracking Timeline:**\n`;

  steps.forEach((step, i) => {
    const done    = i <= statusIdx;
    const current = i === statusIdx;
    timeline += `${done ? icons[i] : '□'} ${labels[i]}
                ${current ? '← *Current*' : ''}\n`;
  });
```

```
  if (order.status === 'cancelled') timeline += `\n❌ **Order Cancelled**`;
  return timeline;
};
```

▸ **Order Tracking Query (Backend)**

```
// Fetch latest or specific order for authenticated user
const orderIdMatch = msg.match(/[a-f0-9]{24}/i);
if (orderIdMatch) {
  const order = await Order.findOne({
    _id: orderIdMatch[0], user: userId
  });
  return res.json({ text: formatOrderTimeline(order) });
}
// No specific ID — show most recent
const orders = await Order.find({ user: userId })
  .sort({ createdAt: -1 })
  .limit(3);
```

4 Cart & Checkout Assistance

Nova supports full conversational cart management. Users can add items, remove items, view their cart summary, and apply coupons — all without leaving the chat window. The current cart state (items, quantities, total) is passed with every API request so the bot always has up-to-date context.

▸ **a) Add Items via Chat**

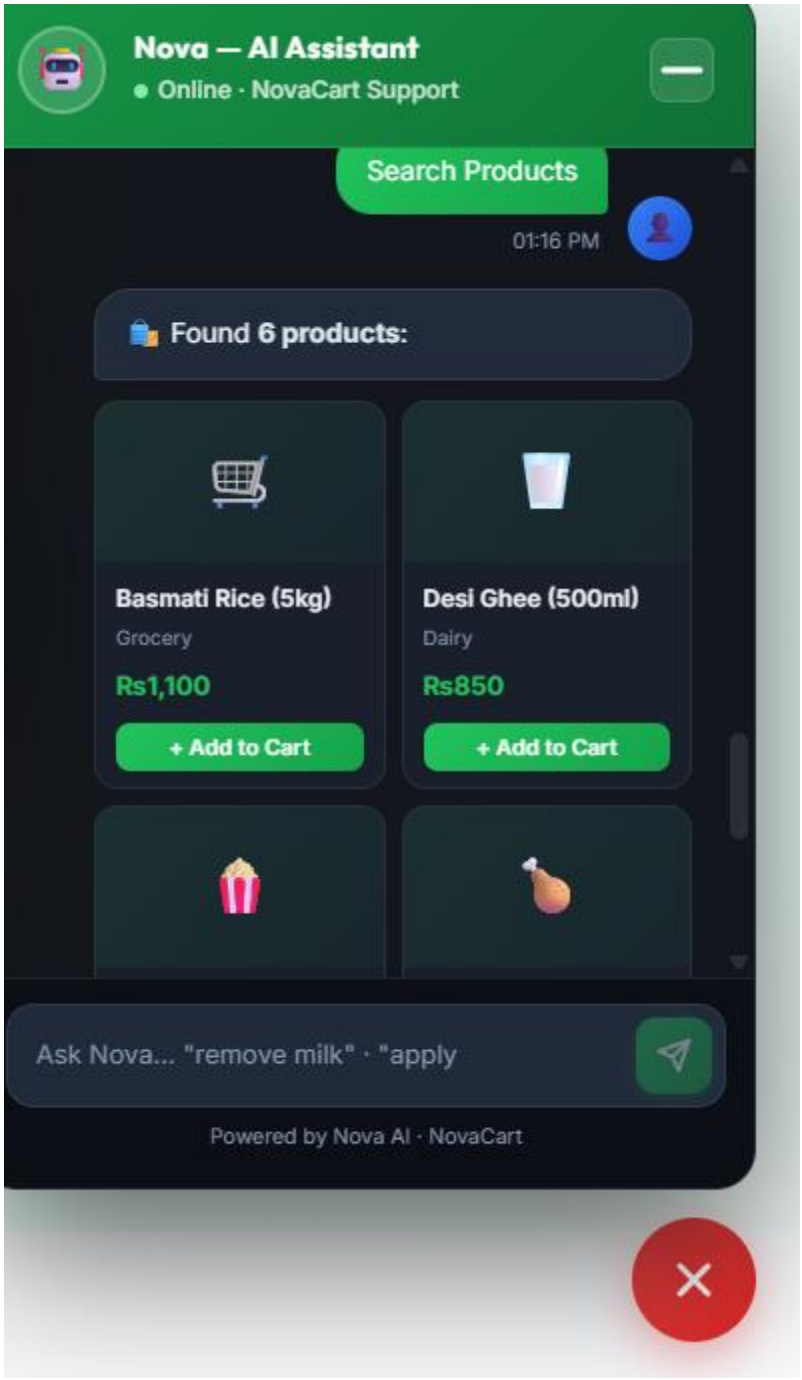
When the user says 'add milk to cart', the bot extracts the product name from the message, searches MongoDB for matching products, and renders interactive product cards with an 'Add to Cart' button directly in the chat bubble.

```
// chatbotController.js — cart_add intent
const products = await Product.find({
  name: { $regex: productName, $options: 'i' }
}).limit(3);

return res.json({
  intent: 'cart_add',
  text: `🛒 Found **${products.length}** product(s). Tap Add to Cart:`,
  products: products.map(p => ({
    _id: p._id, name: p.name, price: p.price,
    image: p.image, category: p.category, stock: p.stock
  })),
  cartAction: 'add'
});
```

▸ **b) Remove Items via Chat**

When the user says 'remove milk from cart', the bot matches the product name against the live cartItems context. If exactly one match is found, a confirmation card (RemoveItemCard) is shown with Remove / Keep buttons. Multiple matches show a selection list (MultiRemoveCard).

```
// chatbotController.js — cart_remove intent
const matched = cartItems.filter(i =>
  i.name.toLowerCase().includes(productName.toLowerCase())
);

if (matched.length === 1) {
  return res.json({
    intent: 'cart_remove_confirm',
    text: `□ Remove **${matched[0].name}** from your cart?`,
    removeItem: {
      productId: matched[0].productId,
      name: matched[0].name,
      price: matched[0].price,
      quantity: matched[0].quantity
    }
  });
}
```

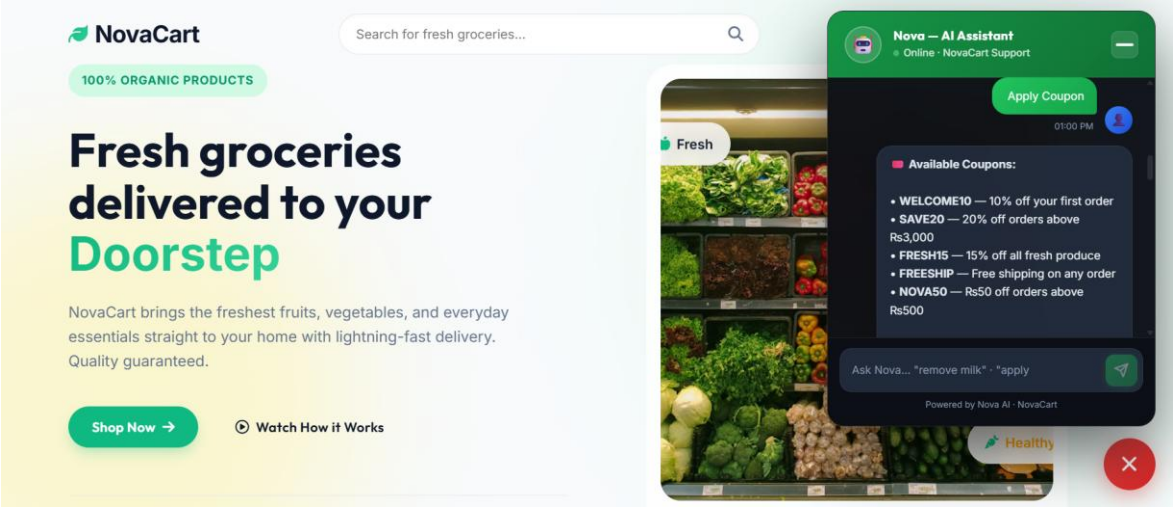


▸ c) Apply Coupons via Chat

Users say 'apply coupon WELCOME10' and the bot validates the code against a defined coupon registry. If the minimum order requirement is met, a CouponCard UI component appears with an 'Apply to Cart' button. Clicking it calls applyDiscount() in CartContext which reflects the discount across the app instantly.

```
// chatbotController.js — valid coupons registry
const VALID_COUPONS = {
  WELCOME10: { type:'percent', value:10, minOrder:0,
```

```
                description:'10% off your first order' },
  SAVE20:     { type:'percent', value:20, minOrder:3000,
                description:'20% off orders above ₨3,000' },
  FRESH15:    { type:'percent', value:15, minOrder:0,
                description:'15% off all fresh produce' },
  FREESHIP:   { type:'shipping',value:150,minOrder:0,
                description:'Free shipping on any order' },
  NOVA50:     { type:'flat',    value:50,  minOrder:500,
                description:'₨50 off orders above ₨500' },
};
```



▸ d) Abandoned Cart Reminders

When the user opens the chatbot after having items in their cart for more than 10 minutes without checking out, Nova automatically displays an AbandonedCartBanner at the top of the chat panel. The banner shows the number of items and subtotal, with a direct Checkout → button.

```
// ChatBot.jsx — Abandoned cart detection (10-minute threshold)
const ABANDONED_THRESHOLD_MS = 10 * 60 * 1000;

useEffect(() => {
  if (isOpen && messages.length === 0) {
    setTimeout(() => {
      if (!abandonedShown.current && cartCount > 0 && lastCartActivity) {
        const timeSince = Date.now() - lastCartActivity;
        if (timeSince >= ABANDONED_THRESHOLD_MS) {
          setShowAbandonedBanner(true);  // triggers AbandonedCartBanner
          abandonedShown.current = true; // show only once per session
        }
      }
    }, 1000);
  }
}, [isOpen]);
```

5 FAQ Automation

Nova can instantly answer the most common customer support questions without any human involvement. The FAQ engine uses keyword matching against a curated knowledge base. When the user's message contains any of the trigger keywords for a topic, the full pre-written answer is returned.

▸ FAQ Topics Covered

- □ Shipping & Delivery — standard/express fees, delivery cities, same-day dispatch cutoff
- □ Return & Refund Policy — 7-day window, perishables exclusion, damaged item process
- □ Payment Methods — Credit/Debit cards, COD, JazzCash, Easypaisa, Bank Transfer
- □ Contact & Support — phone, email, WhatsApp, live chat hours
- □ Available Coupons — lists all active discount codes
- □ Operating Hours — delivery and support availability

▸ FAQ Knowledge Base (Backend)

```
// chatbotController.js — FAQ knowledge base
const FAQ = {
  shipping: {
    keywords: ['shipping','delivery','how long','how fast',
               'days','arrives','arrival'],
    answer: `□ **Shipping Info:**
• Standard Delivery: 3-5 business days — ₨150 flat fee
• Express Delivery:  1-2 business days — ₨300
• Free Shipping on orders above ₨2,000
• Orders placed before 3 PM are dispatched same day!`
  },
  returns: {
    keywords: ['return','refund','exchange','money back',
               'damaged','wrong item'],
    answer: `□ **Return & Refund Policy:**
• 7-day return window from delivery date
• Perishable goods cannot be returned
• Damaged/wrong items? Contact us within 24 hours!`
  },
  payment: {
    keywords: ['payment','pay','credit card','cash','cod',
               'jazzcash','easypaisa'],
    answer: `□ **Payment Methods:**
• Credit/Debit Cards, Cash on Delivery
• JazzCash & Easypaisa (mobile wallets)
• Bank Transfer — HBL, UBL, MCB
• All online payments are 100% secure □`
  }
  // ...contact, coupon, hours topics
};

// FAQ intent match
const detectFAQ = (msg) => {
  const lower = msg.toLowerCase();
  for (const [, data] of Object.entries(FAQ)) {
    if (data.keywords.some(kw => lower.includes(kw)))
      return data.answer;
  }
```

```
  return null;
};
```

▸ **Offline Fallback**

If the backend is unreachable (network error), the frontend ChatAgent.js has a local offlineFallback() function that handles basic queries (shipping, returns, payment, greetings) using simple regex patterns — ensuring the chatbot is never completely unresponsive.

```
// ChatAgent.js — runs in browser when the server is down
const offlineFallback = (message) => {
  const msg = message.toLowerCase();
  if (/ship|deliver/.test(msg))
    return { text: '□ Standard delivery: 3-5 days at ₨150. Free above ₨2,000!'
};
  if (/return|refund/.test(msg))
    return { text: '□ 7-day return window. Refund in 3-5 business days.' };
  return { text: "□ I'm having trouble connecting. Please try again soon!" };
};
```

System Architecture Overview

The chatbot follows a client-server architecture with three main layers:

- Frontend Layer — ChatBot.jsx renders the glassmorphism chat panel, handles user input, displays product cards, and manages cart/coupon interactions via React hooks and context.
- Agent Layer — ChatAgent.js acts as a middleware inside the browser; it formats requests, attaches JWT tokens, calls the REST API, and provides an offline fallback.
- Backend Layer — chatbotController.js on Express/Node.js handles all NLP (intent detection, entity extraction), queries MongoDB (Products, Orders), and returns structured JSON responses.

▸ **API Endpoints**

```
POST  /api/chatbot/query          → Main NLP handler (requires optional JWT)
POST  /api/chatbot/apply-coupon   → Coupon validation endpoint
GET   /api/chatbot/suggestions?q= → Autocomplete suggestions
```

Conclusion

All five core chatbot features have been successfully implemented and integrated into the NovaCart e-commerce platform. The AI agent, Nova, provides a seamless conversational shopping experience — from discovering products with natural language queries to managing the cart, tracking orders, redeeming coupons, and getting instant answers to support questions.

The system is production-ready: it handles authenticated and guest users, degrades gracefully when offline, and is built on a clean, modular architecture that can be extended with machine learning models (e.g., intent classification via Dialogflow or GPT) in future iterations.